

Figure 5: Scaling down a Docker container

Figure 6: The output of the DF command in the NFS client

Dynamically scaling applications up or down in a test environment (TE)

In the above section, we looked at starting applications and then taking them to the running state. This we have done using distributed nodes in a test environment. Based on the scalability requirements described earlier, we can easily scale the applications that are running, up or down, dynamically — without stopping the tests.

Note: We need to also be aware that we are running two containers, one in each of the respective nodes.

In this scenario, we will increase the number of containers to scale up to four instances of the Jmeter application image bundle. Execute the command given below to increase the count of containers to four:

```
# sudo docker service scale stackjmeter_jmeter=4
```

The screen shots in Figure 3 were taken at the same instant, on both the nodes.

Figure 7: NFS mount point common storage

Figure 8: NFS mount of test results and data

Now we can see that two instances of Jmeter are running in each of the nodes. Similar to the above steps, the scaling down of instances can be achieved dynamically without influencing the other running instances. To scale down and go back to the earlier state of only two instances of the Jmeter application bundle, execute the command given below on the master node:

```
# sudo docker service scale stackjmeter_jmeter=2
```

One instance in each node has been brought down, and we can see that only one instance is running in each of the nodes.

Note: In the sections ‘Running applications in the Docker Swarm environment’ and ‘Dynamically scaling applications up or down in a TE’, all the procedures/steps can be executed via SSH with a remote login.

Test data management in Docker

Since tests are running in a distributed and dynamic environment, maintaining the test data across the containers is very important. To avoid generating duplicate data or losing data during creation, tearing down of instances or containers is also critical. This can be achieved through NFS mount and by having a common NFS mount point across the containers. The following are the procedures that can be followed to create and share the NFS mount point.

On NFS server (master node)

```
#sudo apt-get update
#sudo apt-get install nfs-kernel-server
#sudo mkdir /var/nfs/sysconfig -p
#ls -la /var/nfs/sysconfig/
#sudo chown nobody:nogroup /var/nfs/sysconfig
#ls -la /var/nfs/sysconfig/
#sudo systemctl restart nfs-kernel-server
#sudo ufw allow from <IP address of NFS client> to any port nfs
```

So the test data, agent scripts and images are managed in the directories under `/nfs/sysconfig/`, which is `/nfs/sysconfig/sysconfig` (refer to Figure 1 and the first code snippet in the article).

Persistence of test results and logs

Storage of test results and logs is mandatory for any test management, and we need to maintain data persistence even after tearing down the instances and the containers. To achieve this, follow the procedure explained below.

As stated earlier, similar to the above test data management, test logs and test results are stored in the NFS mount. All the test logs and the test results are stored and managed in the directories under `/nfs/sysconfig/`, which is `/nfs/sysconfig/sysdata` (refer to the first code snippet).

Here, all the data is stored in the NFS server (master